

CSE233 Week 11: User Defined Data Types (Enums, Structs, and Classes)

Throughout the course we have been using the predefined data types provided by C++, such as `int`, `double`, and `std::string`. This week, we will discuss the mechanisms provided by C++ for creating our own data types. First, we'll discuss enums, which provide a way to group global constant values. In the past couple weeks we've used arrays, which allow you to create collections of the same data type. Structs and classes allow you to create your own data types with different underlying data types.

Enums

Enums assign human-readable names to integral constants. C++ has two types of enum, the classical C enum, and the more modern enum class. First, let's look at a C enum:

```
enum Day {  
    SUNDAY, MONDAY, TUESDAY,  
    WEDNESDAY, THURSDAY,  
    FRIDAY, SATURDAY  
};
```

Under the hood, the value 0 is assigned to SUNDAY, 1 is assigned to MONDAY, 2 to TUESDAY, and so on. This code below is roughly equivalent:

```
const int SUNDAY = 0;  
const int MONDAY = 1;  
const int TUESDAY = 2;  
const int WEDNESDAY = 3;
```

```
const int THURSDAY = 4;
const int FRIDAY = 5;
const int SATURDAY = 6;
```

We can also assign different values to the enum values:

```
enum Color {
    RED = 40,
    GREEN = -30,
    BLUE = 0,
    YELLOW = 1000
};
```

Enums will automatically start counting up again from the last specified value:

```
enum Direction {
    EAST = 1,
    WEST, // = 2
    SOUTH, // = 3
    NORTH = 10,
    NORTHEAST, // = 11
    // ...
};
```

Most of the time, letting the enum automatically assign values is sufficient.

Enums provide a convenient way to create named constants to represent a group of related values. An enum variable is created by using the enum's name as the type:

```
// enum can be scoped with the enum name
Day sunday = Day::SUNDAY;
// but doesn't have to be
Day monday = MONDAY;
// the underlying integer value can also be used
Day tuesday = 2;
```

Enums can also be used as parameters and return values of functions:

```
Day readDay() {
    std::string day;
    std::cout << "Enter day: ";
```

```

std::cin >> day;
if (day == "Sunday") {
    return Day::SUNDAY;
}
else if (day == "Monday") {
    return Day::MONDAY;
}
else if (day == "Tuesday") {
    return Day::TUESDAY;
}
else if (day == "Wednesday") {
    return Day::WEDNESDAY;
}
else if (day == "Thursday") {
    return Day::THURSDAY;
}
else if (day == "Friday") {
    return Day::FRIDAY;
}
else if (day == "Saturday") {
    return Day::SATURDAY;
}
else {
    // code to handle invalid input would go here
}
}

void printDay(Day day) {
    switch (day) {
        case SUNDAY:
            std::cout << "Sunday";
        case MONDAY:
            std::cout << "Monday";
        case TUESDAY:
            std::cout << "Tuesday";
        case WEDNESDAY:
            std::cout << "Wednesday";
        case THURSDAY:
            std::cout << "Thursday";
        case FRIDAY:
            std::cout << "Friday";
        case SATURDAY:
            std::cout << "Saturday";
    }
}

```

One limitation of enums is that they are just integers underneath, which means that invalid values can be assigned to an enum variable very easily:

```
// -1 is not a valid day, but the compiler may not prevent this
Day day = -1;
```

C++11 added enum classes to deal with this. An enum class is like a normal enum, but adds several limitations:

- The underlying integer value can no longer be used to access the enum's values
- The enum values are always scoped
- Invalid values can no longer be assigned to enums

Here is the `Day` enum as an enum class:

```
enum class Day {
    SUNDAY, MONDAY, TUESDAY,
    WEDNESDAY, THURSDAY,
    FRIDAY, SATURDAY
};
```

Now, the only way to assign an enum class value is to scope it.

```
// still legal
Day sunday = Day::SUNDAY;
// not legal
// Day monday = MONDAY;
// the value must be scoped
Day monday = Day::MONDAY;
// also no longer legal
// Day tuesday = 2
// again, needs to be scoped
Day tuesday = Day::TUESDAY;
```

Enum classes are still integers underneath, and can otherwise be used in the exact same way as C enums.

Structs

A struct (short for structure) is a composite data type. A struct consists of member variables, called fields, that are stored within each instance of the struct. Below is an example struct:

```
struct RGBColor {
    int red;
```

```
    int green;  
    int blue;  
};
```

A struct declaration consists of the `struct` keyword, followed by the name of the struct and a block. The block contains the struct's fields. The struct ends with a semicolon. The above struct has three fields, which are all integers. RGB (red, green blue) color is the predominant way of storing colors in digital formats, with values between 0 and 255 assigned to each color channel to form a single pixel. However, the fields of a struct do not have to have the same data type:

```
struct Student {  
    std::string firstName;  
    std::string lastName;  
    int creditHours;  
    double gpa;  
};
```

The student struct consists of two strings for a student's first and last name, an int for the number of credit hours completed, and a double for the student's GPA. To declare a struct variable, we use the name of the struct as any other data type:

```
Student student;
```

We can access the fields of the struct using the dot operator:

```
Student student;  
student.firstName = "John";  
student.lastName = "Doe";  
student.creditHours = 60;  
student.gpa = 3.1;
```

When creating the struct, we can also use an initializer list to provide initial values to the struct variable's fields. The initializer list is a set of curly braces with values separated by commas. The values passed to the initializer list are assigned to the struct's fields in the order they appear in the struct declaration.

```
// legal, firstName is "John", the rest are unassigned
```

```

Student student1 = {"John"};
// legal, firstName is "John" and lastName is "Doe"
Student student2 = {"John", "Doe"};
// legal, firstName is "John", lastName is "Doe", creditHours is 60
Student student3 = {"John", "Doe", 60};
// legal, firstName is "John", lastName is "Doe",
// creditHours is 60, gpa is 3.1
Student student4 = {"John", "Doe", 60, 3.1};
// illegal, will try to store 3.1 in an int (creditHours),
// which can't be done
Student student5 = {"John", "Doe", 3.1};

```

For the most part, structs can be used like any other primitive type. They can be used as both return types and parameters to functions. You can also create arrays of structs:

```

#include <iostream>

struct Student {
    std::string firstName;
    std::string lastName;
    int creditHours;
    double gpa;
};

void printStudent(const Student &student) {
    std::cout << "Name:    " << student.firstName
               << " " << student.lastName << "\n";

    std::cout << "Credits: " << student.creditHours << "\n";
    std::cout << "GPA:      " << student.gpa << "\n";
}

int main() {
    Student students[10];

    std::cout << "Enter student information: \n";
    for (int i = 0; i < 10; ++i) {
        std::cout << "Enter first name: ";
        std::cin >> student[i].firstName;
        std::cout << "Enter last name: ";
        std::cin >> student[i].lastName;
        std::cout << "Enter credit hours: ";
        std::cin >> student[i].creditHours;
        std::cout << "Enter GPA: ";
        std::cin >> student[i].gpa;
    }

    std::cout << "You entered\n";
}

```

```

    for (int i = 0; i < 10; ++i) {
        std::cout << "\nStudent " << i + 1 << " ----- \n";
        printStudent(students[i]);
        std::cout << "----- \n";
    }

    return 0;
}

```

The only thing we can't do with structs is use operators like +, -, ==, <, >, etc. on them, or directly print them using `std::cout <<` or similar. Next week, we will discuss operator overloading, which will allow us to use these operators for structs.

Classes

A class is very similar to a struct (in C++, they are basically the same, but programmers use them differently). Structs are used to store simple data with few or no operations. All data in a struct is public, by default. In a class, data and member functions are private, by default. The `public:` region specifier must be used to make fields and member functions usable outside the class. Below is a basic class:

```

class Student {
public:
    std::string firstName;
    std::string lastName;
    int creditHours;
    double gpa;
};

```

In this case, we've made all the fields public, so this class is basically the same as a struct. In object-oriented design, we typically make the data fields of a class private. This means that the fields of the class can no longer be accessed directly by objects when we attempt to use them:

```

class Student {
private:
    std::string firstName;
    std::string lastName;
    int creditHours;
    double gpa;
};

```

```
int main() {
    Student student;
    // can't do this, firstName is private
    student.firstName = "John";
}
```

In the following sections, we'll discuss the categories of functions that are defined for classes to allow us to use the data stored by the class.

Constructors

A constructor is special function that classes have to create new objects. Constructors are member functions with no return type and the same name as the class. The constructor is called whenever an object is created. Functions, like the constructor, can be defined directly within the class.

```
class Student {
// public region specifier, anything after this is public
public:
    // default constructor, does nothing
    Student() {}
    // constructor that allows us to set the fields
    // due to width constraints on the PDF, poor names are being used
    // for the parameters in this example
    // the full code is provided in a C++ file this week with better n
    Student(std::string f, std::string l, double g, int c) {
        // the names of the member variables are used directly
        // directly inside of class member functions
        // no dot operator or this/self reference is required
        firstName = f;
        lastName = l;
        creditHours = c;
        gpa = g;
    }
// private region specifier, ends the public region
// anything after this is now private
private:
    std::string firstName;
    std::string lastName;
    int creditHours;
    double gpa;
};
```

Or, more typically in C++, they are defined outside the class after they are declared. The function must be scoped to the class by putting the

class name before the name of the function, separated by double colons.

```
class Student {
public:
    // default constructor, does nothing
    Student() {}
    // constructor that allows us to set the fields
    Student(std::string f, std::string l, double g, int c);
private:
    std::string firstName;
    std::string lastName;
    int creditHours;
    double gpa;
};
// defined outside the class. the constructor is associated with the c
// thanks to Student:: being placed before the name
Student::Student(std::string f, std::string l, double g, int c) {
    firstName = f;
    lastName = l;
    creditHours = c;
    gpa = g;
}
```

The constructor is called by passing arguments to the object being created:

```
int main() {
    // create a student object with the constructor
    Student johnDoe("John", "Doe", 3.5, 24);
    // create another student object
    Student janeDoe("Jane", "Doe", 3.5, 24);

    return 0;
}
```

While this allows us to create objects with data, it doesn't allow us to access the data. The next category of member function is used to read and modify the properties of a class.

Accessors

An accessor is a function used to access the values of a class's fields. There are two primary categories of member function: getters and setters. Getters are used to read the values of a field. Setters are used

to modify the values of fields. There is no rule for how to name these functions, but getters typically start with `get` and setters typically start with `set`. In some cases, getters and setters can also just be the name of a field.

```
class Student {
public:
    Student() {}
    Student(std::string f, std::string l, double g, int c);
    // getters
    std::string getFirstName() const;
    std::string getLastName() const;
    int getCreditHours() const;
    double getGpa() const;
    // setters
    void setFirstName(const std::string& name);
    void setLastName(const std::string& name);
    void setCreditHours(int credits);
    void setGpa(double grade);
private:
    std::string firstName;
    std::string lastName;
    int creditHours;
    double gpa;
};

Student::Student(std::string f, std::string l, double g, int c) {
    firstName = f;
    lastName = l;
    creditHours = c;
    gpa = g;
}

// remember, the name of the class goes on the name of the function
std::string Student::getFirstName() const {
    return firstName;
}

std::string Student::getLastName() const {
    return lastName;
}

int Student::getCreditHours() const {
    return creditHours;
}

double Student::getGpa() const {
    return gpa;
}

void Student::setFirstName(const std::string& name) {
```

```

        firstName = name;
    }

    void Student::setLastName(const std::string& name) {
        lastName = name;
    }

    void Student::setCreditHours(int credits) {
        creditHours = credits;
    }

    void Student::setGpa(double grade) {
        gpa = grade;
    }

```

Notice that the getters are marked as `const`. Placing `const` after the function signature indicates that the member function does not modify the state of the class. Since the getters are only returning copies of the member variables, they do not have any chance of modifying the object calling the function's state. Meanwhile, the setters do modify the values of the object that calls them, so they cannot be marked `const`. Here are some examples of the get/set functions in use:

```

int main() {
    Student student1("John", "Doe", 3.5, 24);
    Student student2("Jane", "Doe", 3.5, 24);

    std::cout << "Before calling setters\n";

    std::cout << "Student 1: \n";
    std::cout << "    " << student1.getFirstName() << " "
        << student1.getLastName() << "\n";
    std::cout << "    "
        << student1.getGpa() << " GPA\n";
    std::cout << "    "
        << student1.getCreditHours() << " Credit Hours\n";

    std::cout << "\nStudent 2: \n";
    std::cout << "    " << student2.getFirstName() << " "
        << student2.getLastName() << "\n";
    std::cout << "    "
        << student2.getGpa() << " GPA\n";
    std::cout << "    "
        << student2.getCreditHours() << " Credit Hours\n";

    student1.setFirstName("Mike");
    student1.setLastName("Smith");
    student1.setGpa(3.6);
}

```

```

student1.setCreditHours(12);

student2.setFirstName("Sam");
student2.setGpa(500);

std::cout << "\nAfter calling setters\n";
    std::cout << "Student 1: \n";
std::cout << "    " << student1.getFirstName() << " "
    << student1.getLastName() << "\n";
std::cout << "    "
    << student1.getGpa() << " GPA\n";
std::cout << "    "
    << student1.getCreditHours() << " Credit Hours\n";

std::cout << "\nStudent 2: \n";
std::cout << "    " << student2.getFirstName() << " "
    << student2.getLastName() << "\n";
std::cout << "    "
    << student2.getGpa() << " GPA\n";
std::cout << "    "
    << student2.getCreditHours() << " Credit Hours\n";

return 0;
}

```

Output:

```

Before calling setters
Student 1:
    John Doe
    3.5 GPA
    24 Credit Hours

Student 2:
    Jane Doe
    3.5 GPA
    24 Credit Hours

After calling setters
Student 1:
    Mike Smith
    3.6 GPA
    12 Credit Hours

Student 2:
    Sam Doe
    500 GPA
    24 Credit Hours

```

As can be seen, implementing a simple set function may be inadequate. GPAs should be between 0 and 4, but we were able to set it to 500. This is one reason not to make the data members of a class public, as it makes it easy to set them to invalid values. We can add a guard to the setter for the GPA to ensure its value is valid (this check should also be added to the constructor):

```
void Student::setGpa(double grade) {  
    if (grade < 4 && grade > 0) {  
        gpa = grade;  
    }  
    else if (grade > 4) {  
        gpa = 4;  
    }  
    else {  
        gpa = 0;  
    }  
}
```

Other Member Functions

Classes can have other member functions than constructors and accessors. For example, we can add member functions that return the student's full name, updates their GPA and credit hours, and

```
class Student {  
public:  
    Student() {}  
    Student(std::string f, std::string l, double g, int c);  
    // getters  
    std::string getFirstName() const;  
    std::string getLastName() const;  
    int getCreditHours() const;  
    double getGpa() const;  
    // setters  
    void setFirstName(const std::string& name);  
    void setLastName(const std::string& name);  
    void setCreditHours(int credits);  
    void setGpa(double grade);  
    // other functions  
    // returns the name in "First Last" format  
    std::string getFullName() const;  
    // allows users to specify the output format of the name  
    // looks for {first} and {last} in the format string  
    // for example, format = "{last}, {first}" outputs a name  
    // as "Last, First"
```

```

        std::string getFullName(const std::string& format) const;
        void completeClass(double grade, int credits);
private:
        std::string firstName;
        std::string lastName;
        int creditHours;
        double gpa;
};
// other member functions omitted for brevity
std::string Student::getFullName() const {
    return firstName + " " + lastName;
}

std::string Student::getFullName(const std::string& format) const {
    std::string output = format;
    // find "{first}" and replace it with firstName
    int firstPos = output.find("{first}");
    if (firstPos != std::string::npos) {
        output.replace(firstPos, 7, firstName);
    }

    // find "{last}" and replace it with lastName
    int lastPos = output.find("{last}");
    if (lastPos != std::string::npos) {
        output.replace(lastPos, 6, lastName);
    }

    return output;
}

void Student::completeClass(double grade, int credits) {
    // convert grade to gradePoint
    double gradePoint;
    // A is 4.0
    if (grade >= 90) {
        gradePoint = 4;
    }
    // B is 3.0
    else if (grade >= 80) {
        gradePoint = 3;
    }
    // C is 2.0
    else if (grade >= 70) {
        gradePoint = 2;
    }
    // D is 1.0
    else if (grade >= 60) {
        gradePoint = 1;
    }
    // F is 0
    else {

```

```

        gradePoint = 0;
    }
    // now, use weighted average calculation to determine
    // overall GPA based on the old GPA and new class
    int newCredits = credits + creditHours;
    // (double)newCredits temporarily converts newCredits to a double
    // that way, it performs floating point division instead of integer
    // also doing some funky formatting so it fits on the page
    double weightedGpa
        = (creditHours / (double)newCredits) * gpa;
    double weightedGradePoint
        = (credits / (double)newCredits) * gradePoint;
    gpa = weightedGpa + weightedGradePoint;
    creditHours = newCredits;
}

```

Example of these new member functions being used:

```

int main() {
    Student student("John", "Doe", 3.5, 24);
    std::cout << student.getFullName() << "\n";
    std::cout
        << student.getFullName("Student: {last}, {first}") << "\n";
    std::cout << student.getGpa() << " GPA\n";
    std::cout << student.getCreditHours() << " Credit Hours\n";

    // simulate student completing three classes
    // and getting 2 As and a B
    student.completeClass(95, 3);
    student.completeClass(87, 3);
    student.completeClass(91, 3);

    // print updated gpa and credit hours
    std::cout << "\nUpdated credit hours and GPA\n";
    std::cout << student.getGpa() << " GPA\n";
    std::cout << student.getCreditHours() << " Credit Hours\n";

    return 0;
}

```

Output:

```

John Doe
Student: Doe, John
3.5 GPA
24 Credit Hours

Updated credit hours and GPA

```

Comparing Structs and Classes

Technically speaking, there is almost no difference between a struct and class in C++. The main difference to most programmers is that everything is public by default in a struct, while everything is private by default in a class. There are some low-level differences when dealing with backwards compatibility with C code, but for the sake of this course a struct and a class can do the same things. Despite this, C++ programmers generally use structs and classes in different ways. When a programmer-defined data type is only being used to store simple data with few or no operations, structs are typically used. When the inner data is more complex or has more complex relations (where private visibility is needed), classes are preferred. Classes are also typically used when a type has many operations defined for it. If the only operations defined are get/set functions, then you may be better off using a struct and making everything public.

Conclusion

This week we discussed programmer defined data types in C++. Enums are used to declare named constants, typically those that can be grouped together. Structs and classes are mechanically the same in C++, but used in different ways. Structs are used to store plain data with few or no operations. The data in structs is public (accessible by objects outside the struct). Classes have private access by default, meaning that the data and member functions cannot be accessed by objects declared outside the class. Classes typically implement constructors to allow the initial values of an object's fields to be set when the class is created. Accessor functions are used to read and modify the values of an object's fields. Classes can also implement other member functions for various other operations.